



Module: Développer en back-end

M205-complément2- Envoi d'email, notification et évènement



Filière: Développement digital – option web full stack

Cours réalisé par :

Ouafae El Maudni & Asmae YOUALA

ISTA AL ADARISSA

PLAN

- A. Envoi d'email**
- B. Notification**
- C. Introduction aux évènements**
- D. Utiliser des notifications avec des événements dans Laravel**

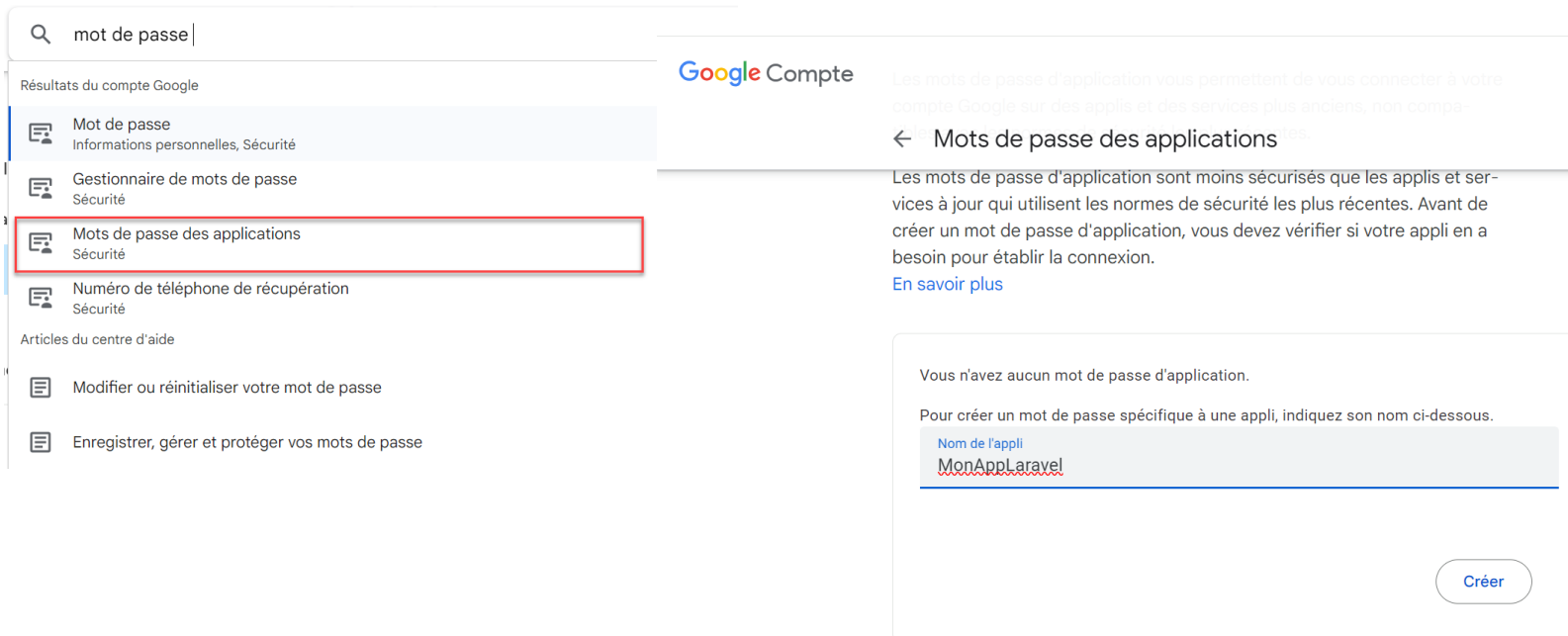
A. Envoi d'email

1. Configuration

- L'envoi de mails est une tâche courante dans les applications web, et Laravel facilite cette opération grâce à son système de notification et de mail intégré.
- Avant de commencer l'envoi d'e-mails sous Laravel, il est essentiel de configurer un driver et un serveur de messagerie.
- Le driver le plus couramment utilisé est SMTP, bien qu'il existe d'autres options telles que Mailgun, Postmark et Amazon SES.

Configuration d'un serveur SMTP Gmail:

Pour configurer l'envoi d'email via un compte gmail, il faut configurer un mot de passe d'application au niveau du compte Gmail et fournir les informations nécessaires au niveau du fichier **.env**:



Google Compte

Les mots de passe d'application vous permettent de vous connecter à votre compte Google sur des applis et des services plus anciens, non compatibles.

← Mots de passe des applications

Les mots de passe d'application sont moins sécurisés que les applis et services à jour qui utilisent les normes de sécurité les plus récentes. Avant de créer un mot de passe d'application, vous devez vérifier si votre appli en a besoin pour établir la connexion.

[En savoir plus](#)

Vous n'avez aucun mot de passe d'application.

Pour créer un mot de passe spécifique à une appli, indiquez son nom ci-dessous.

Nom de l'appli
MonAppLaravel

[Créer](#)

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME="xxxxxxxxx@gmail.com"
MAIL_PASSWORD="xxxxxxxxxxxxx"
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS="admin@example.com"
MAIL_FROM_NAME="${APP_NAME}"
```

A. Envoi d'email

1. Configuration

- Différents outils sont disponibles pour installer et tester localement un serveur e-mail, notamment HELO, Mailtrap, Mailpit et MailHog.
- **Configuration de de MailHog comme serveur local**

Voici un exemple de configuration pour utiliser le serveur SMTP local de MailHog :

```
MAIL_MAILER=smtp
MAIL_HOST=localhost
MAIL_PORT=1025
MAIL_USERNAME=null
MAIL_PASSWORD=null
MAIL_ENCRYPTION=null
MAIL_FROM_ADDRESS="admin@example.com"
MAIL_FROM_NAME="${APP_NAME}"
```

Téléchargez MailHog via ce [lien](#), démarrez-le et accédez localement à son tableau de bord via : <http://localhost:8025>

A. Envoi d'email

2. Création d'une Classe de Mail

Vous pouvez créer une classe de mail en utilisant la commande artisan suivante :

Exemple:

```
php artisan make:mail TestMail
```

Cette commande crée une nouvelle classe de mail dans app/Mail/TestMail.php.

```
class TestMail extends Mailable
{
    use Queueable, SerializesModels;
    public $nom;
    public function __construct($nom)
    {
        $this->nom = $nom;
    }

    public function envelope(): Envelope
    {
        return new Envelope(
            subject: 'Test Mail',
        );
    }

    public function content(): Content
    {
        return new Content(
            view: 'mails.welcome',
            with:[
                'nom'=>$this->nom
            ]
        );
    }

    /**
     * Get the attachments for the message.
     *
     * @return array<int, \Illuminate\Mail\Mailables\Attachment>
     */
    public function attachments(): array
    {
        return [];
    }
}
```

- La méthode **envelope** renvoie un objet *Illuminate\Mail\Mailables\Envelope* qui définit le sujet et, parfois, les destinataires du message.
- La méthode **content** renvoie un objet *Illuminate\Mail\Mailables\Content* qui définit le modèle **Blade** qui sera utilisé pour générer le contenu du message.

A. Envoi d'email

3. Création de la Vue pour le Mail

Ensuite, créez une vue pour le mail dans `resources/views/emails/welcome.blade.php`.

Exemple:

```
<!DOCTYPE html>
<html>

<head>
    <title>Bienvenue sur notre plateforme</title>
</head>
<body>
    <h1>Bienvenue, {{$nom}} !</h1>
    <p>Nous sommes ravis de vous accueillir sur notre plateforme. Si vous avez des questions,
n'hésitez pas à nous contacter.</p>
</body>
</html>
```

A. Envoi d'email

4.Envoi d'un Mail

Vous pouvez envoyer un mail en utilisant la façade Mail fournie par Laravel

Exemple:

Supposons que vous ayez un contrôleur appelé UserController et que vous souhaitiez envoyer un mail de bienvenue après l'inscription d'un utilisateur:

```
namespace App\Http\Controllers;

use App\Mail\TestMail;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Mail;

class UserController extends Controller
{
    public function register(Request $request)
    {
        // Validation et création de l'utilisateur
        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => bcrypt($request->password),
        ]);

        // Envoi du mail de bienvenue
        Mail::to($user->email)->send(new TestMail($user->nom));

        return response()->json(['message' => 'User registered and mail sent!']);
    }
}
```

A. Envoi d'email

5. Consultation de l'email reçu :

A travers le dashboard de MailHog <http://localhost:8025>, on peut consulter les emails reçus

 MailHog



 Connected

Inbox (1)

 Delete all messages

Jim

Jim is a chaos monkey.
[Find out more at GitHub.](#)

Enable Jim

From Laravel <admin@example.com>

Subject **Test Mail**

To ahmed.alaoui@gmail.com

HTML Plain text Source

Bienvenue, Ahmed Alaoui !

Nous sommes ravis de vous accueillir sur notre plateforme. Si vous avez des questions, n'hésitez pas à nous contacter.

B. Les notifications en Laravel

- Les notifications dans Laravel sont un moyen pratique d'envoyer des messages via différents canaux (**email, SMS, base de données**, etc.).
- Dans Laravel, on peut choisir d'envoyer des emails ou d'utiliser les notifications en fonction de plusieurs critères liés à la nature du message, à l'audience, et à la manière dont le message sera reçu et traité par les utilisateurs.

Quand Utiliser Mails et Notifications

- **Emails** sont préférables pour des communications formelles, détaillées, et des informations qui doivent être archivées.
- **Notifications Laravel** sont idéales pour des alertes rapides, des messages courts, et des communications multi-canal instantanées.

- Voici un exemple pour envoyer une notification par email lors de l'authentification d'un utilisateur.

▪ 1- Créer une Notification

`php artisan make:notification TestNotification`

Cela va générer un fichier de notification sous le répertoire **app/Notifications/TestNotification.php**

B. Les notifications en Laravel

- Chaque classe de notification contient une méthode `via` et un nombre variable de méthodes de création de messages, telles que **`toMail`** ou **`toDatabase`** ou **`toVonage`**, qui convertissent la notification en message adapté à ce canal particulier.

Configurer l'envoi par email:

```
class TestNotification extends Notification
{
    use Queueable;
    /**
     * Create a new notification instance.
     */
    public function __construct()
    { }

    /**
     * Get the notification's delivery channels.
     *
     * @return array<int, string>
     */
    public function via(object $notifiable): array
    {
        return ['mail'];
    }

    /**
     * Get the mail representation of the notification.
     */
    public function toMail(object $notifiable): MailMessage
```

```

    {
        return (new MailMessage)
            ->line('Vous vous êtes connecté avec succès à
votre compte.')
            ->action('Notification Action', url('/'))
            ->line('Thank you of using our application!');
    }

    /**
     * Get the array representation of the notification.
     *
     * @return array<string, mixed>
     */
    public function toArray(object $notifiable): array
    {
        return [
            //
        ];
    }
}
```

B. Les notifications en Laravel

- Envoyer la Notification

- Avant d'envoyer la notification:

Configurer le Modèle concerné par les Notifications en y ajoutant le trait `use Notifiable` :

```
class User extends Authenticatable
{
    use HasFactory, Notifiable;
    ...}

```

- Envoi de la notification:

L'instance du modèle configuré ainsi, pourra appeler la méthode **notify** fournie par ce trait;

```
use App\Notifications\TestNotification;
$user->notify(new TestNotification());

```

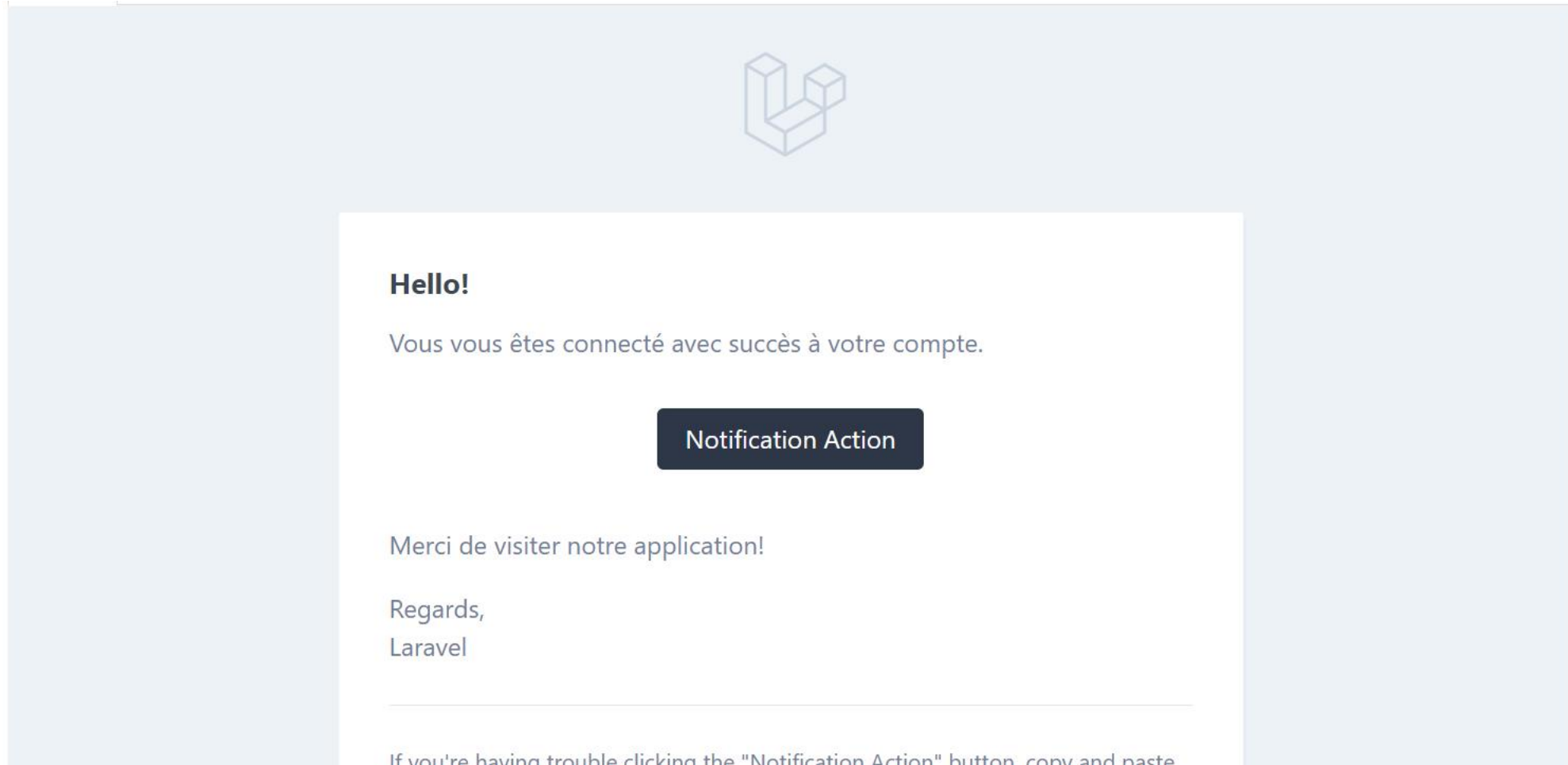
Alternativement, vous pouvez envoyer des notifications via la façade **Notification**. Cette approche est utile lorsque vous devez envoyer une notification à plusieurs entités notifiables telles qu'un ensemble d'utilisateurs. Pour envoyer des notifications à l'aide de la façade, transmettez toutes les entités notifiables et l'instance de notification à la méthode **send**:

```
use Illuminate\Support\Facades\Notification;
Notification::send($users, new TestNotification());

```

B. Les notifications en Laravel

- Notification reçue:



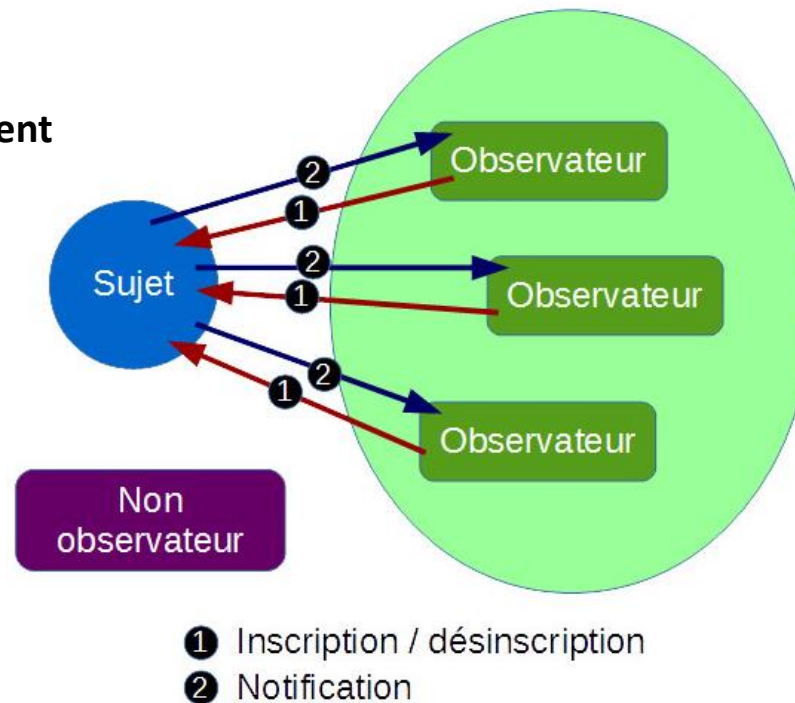
C. Introduction aux évènements

- Un événement (ou "event" en anglais) est un mécanisme permettant de signaler qu'une certaine action ou condition s'est produite dans l'application, et de déclencher des réponses à cette action.
- Les événements sont une partie du système de gestion des événements de Laravel, qui permet de découpler différentes parties de l'application, rendant le code plus modulaire et plus facile à maintenir.

Principe:

- On a un sujet et des observateurs. Les observateurs doivent s'inscrire (on dit aussi s'abonner) auprès du sujet. Lorsque le sujet change d'état il notifie tous ses abonnés. Un observateur peut aussi se désinscrire, il ne recevra alors plus les notifications.

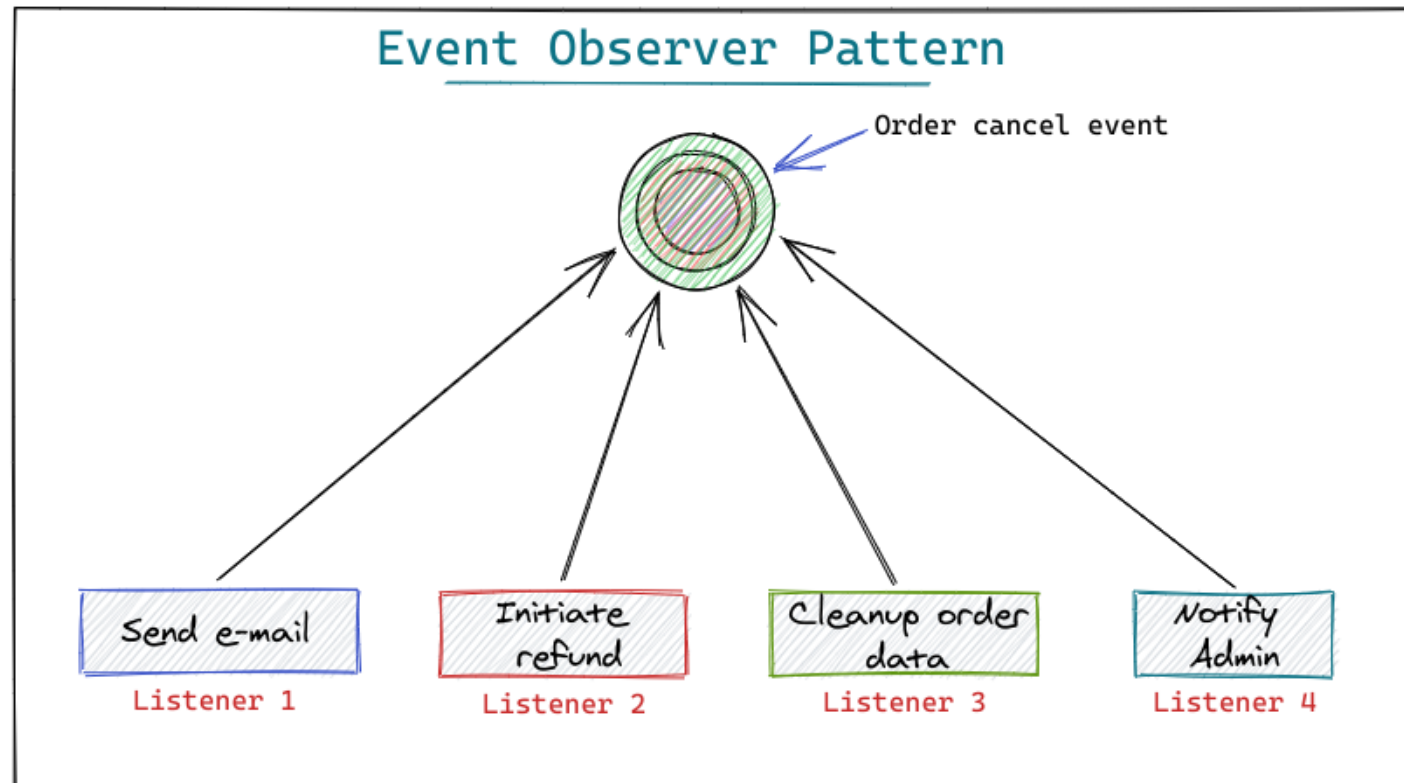
En laravel, le sujet représente l'évènement: **event**
Et les observateurs sont les **listeners**



C. Introduction aux évènements

Exemple:

- **Traitement d'une annulation d'une commande** : Lorsqu'une commande est annulée sur un site e-commerce, cela peut déclencher un événement qui initiera un ensemble d'actions : envoyer un email au client, traiter le remboursement, mettre à jour les données de la commande, et notifier l'administrateur



D. Utiliser des notifications avec des événements dans Laravel

- **Exemple:** Dans cet exemple on va créer deux écouteurs (listeners) pour un événement qui se produit lorsqu'un étudiant est ajouté. L'un des écouteurs enverra un email à l'étudiant nouvellement ajouté, et l'autre enregistrera cette action dans une table d'audit dans la base de données.

Soit la base de données suivante:

Student(id, username,password,name,email)

AuditLog(id, description)

1. Créer l'Événement

php artisan make:event StudentAdded

L'exécution de cette commande permettra de générer une classe sous le répertoire **app/Events/StudentAdded.php**

```
class StudentAdded
{
    use Dispatchable, InteractsWithSockets, SerializesModels;
    /**
     * Create a new event instance.
     */
    public $student;

    public function __construct(Student $student)
    {
        $this->student = $student;
    }
    /**
     * Get the channels the event should broadcast on.
     */
    *
    * @return array<int, \Illuminate\Broadcasting\Channel>
    */
    public function broadcastOn(): array
    {
        return [
            new PrivateChannel('channel-name'),
        ];
    }
}
```

D. Utiliser des notifications avec des événements dans Laravel

- Créer les Écouteurs

2. Créer la notification StudentAddedNotification

php artisan make:notification StudentAddedNotification

```
public function toMail(object $notifiable): MailMessage
{
    return (new MailMessage)
        ->line('Inscription réussie!')
        ->action('Explorez nos cours', url('/cours'))
        ->line('Thank you for using our application!');
}
```

3. Créer un écouteur pour envoyer un email à l'étudiant en l'associant à l'évènement StudentAdded

php artisan make:listener SendStudentAddedEmail --event=StudentAdded

- Dans **app/Listeners/SendStudentAddedEmail.php**

```
class SendStudentAddedEmail
{
    /**
     * Create the event listener.
     */
    public function __construct()
    {
        //
    }

    /**
     * Handle the event.
     */
    public function handle(StudentAdded $event): void
    {
        $student = $event->student;
        $student->notify(new StudentAddedNotification());
    }
}
```


D. Utiliser des notifications avec des événements dans Laravel

- Créer les Écouteurs

4. Créer un écouteur pour enregistrer l'ajout dans un log d'audit dans la base de données :

php artisan make:listener LogStudentAddedToDatabase --event=StudentAdded

```
class LogStudentAddedToDatabase
{
    /**
     * Create the event listener.
     */
    public function __construct()
    {
        //
    }

    /**
     * Handle the event.
     */
    public function handle(StudentAdded $event):
    void
    {
        $student = $event->student;
        // Enregistrer l'action dans la table
        AuditLog
        AuditLog::create([
            'description' => "Étudiant ajouté :
            {$student->name} ({$student->email})",
        ]);
    }
}
```

D. Utiliser des notifications avec des événements dans Laravel

- Enregistrer les Écouteurs

Laravel 10 ou inférieur:

- Enregistrez les écouteurs dans le fichier **app/Providers/EventServiceProvider.php** :

```
protected $listen = [  
    StudentAdded::class => [  
        SendStudentAddedEmail::class,  
        LogStudentAddedToDatabase::class,  
    ],  
];
```

- **Pour Laravel 11:**
- Par défaut, Laravel 11 trouvera et enregistrera automatiquement vos écouteurs d'événements en analysant le répertoire **Listeners** de votre application. Lorsque Laravel trouve une méthode de classe d'écoute commençant par **handle** ou **__invoke**, Laravel enregistre ces méthodes en tant **qu'écouteurs d'événement** pour l'événement indiqué dans la signature de la méthode
- La commande **event:list** peut être utilisée pour lister tous les listeners enregistrés dans votre application :

php artisan event:list

D. Utiliser des notifications avec des événements dans Laravel

- Déclencher l'Événement lors de l'Ajout d'un Étudiant

///Code du contrôleur.....

```
$student = Student::create($request->all());  
event(new StudentAdded($student));
```

.....

- NB: Configurer le Modèle Student pour les Notifications: Assurez-vous que votre modèle **Student** utilise le trait **Notifiable** *use HasFactory, Notifiable;*
- Après avoir ajouté un nouveau student, une notification (sous forme d'email) a été envoyé à l'adresse de l'inscrit et une nouvelle ligne a été ajouté à la table audit_logs

Serveur : 127.0.0.1 » Base de données : tp14 » Table : audit_logs

Parcourir Structure SQL Rechercher Insérer Exporter Importer Privileges Opérations Suivi Déclencheurs

✓ Affichage des lignes 0 - 0 (total de 1, traitement en 0,0007 seconde(s).)

SELECT * FROM `audit\_logs`

Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

Tout afficher Nombre de lignes : 25 Filtrer les lignes: Chercher dans cette table

Options supplémentaires

	id	description	created_at	updated_at
☐ Éditer ☐ Copier ☐ Supprimer	1	Étudiant ajouté : alaoui2 (a.aloui2@gmail.com)	2024-05-26 21:32:38	2024-05-26 21:32:38

